Setup: Graph And Utilities


```
import time
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from collections import deque
```

```
import time
import heapq

graph = {
    'Workstation': {'Firewall': 2, 'Router': 3},
    'Firewall': {'Server': 4, 'IDS': 5},
    'Router': {'Database': 6},
    'Server': {'CriticalAsset': 2},
    'IDS': {'CriticalAsset': 3},
    'Database': {'CriticalAsset': 1},
    'CriticalAsset': {}
}

start_node = 'Workstation'
goal_node = 'CriticalAsset'
```

HEURISTIC VALUES

```
# heuristic values for A* and Hill Climbing
heuristic = {
    'Workstation': 7,
    'Firewall': 6,
    'Router': 5,
    'Server': 3,
    'IDS': 4,
    'Database': 2,
    'CriticalAsset': 0
}
```

BFS

```
import time

def bfs(graph, start, goal):
    start_time = time.time()
    queue = [(start, [start])]
    expanded = 0
    while queue:
        node, path = queue.pop(0)
        expanded += 1
        if node == goal:
            exec_time = time.time() - start_time
            return path, len(path)-1, expanded, exec_time
        for neighbor in graph[node]:
            if neighbor not in path:
                queue.append((neighbor, path + [neighbor]))
    return None
```

DFS

```
def dfs(graph, start, goal):
    start_time = time.time()
    stack = [(start, [start])]
    expanded = 0
    while stack:
        node, path = stack.pop()
        expanded += 1
        if node == goal:
            exec_time = time.time() - start_time
```

```
            return path, len(path)-1, expanded, exec_time
        for neighbor in graph[node]:
            if neighbor not in path:
                stack.append((neighbor, path + [neighbor]))
    return None
```

Show command palette (Ctrl+Shift+P)

Uniform Cost Search (UCS)

```
import heapq

def ucs(graph, start, goal):
    start_time = time.time()
    pq = [(0, start, [start])]
    expanded = 0
    while pq:
        cost, node, path = heapq.heappop(pq)
        expanded += 1
        if node == goal:
            exec_time = time.time() - start_time
            return path, cost, expanded, exec_time
        for neighbor, weight in graph[node].items():
            if neighbor not in path:
                heapq.heappush(pq, (cost+weight, neighbor, path+[neighbor]))
    return None
```

A* Search

```
def a_star(graph, start, goal):
    start_time = time.time()
    pq = [(heuristic[start], 0, start, [start])]
    expanded = 0
    while pq:
        est, cost, node, path = heapq.heappop(pq)
        expanded += 1
        if node == goal:
            exec_time = time.time() - start_time
            return path, cost, expanded, exec_time
        for neighbor, weight in graph[node].items():
            if neighbor not in path:
                g = cost + weight
                f = g + heuristic[neighbor]
                heapq.heappush(pq, (f, g, neighbor, path+[neighbor]))
    return None
```

Hill Climbing

```
def hill_climbing(graph, start, goal):
    start_time = time.time()
    current = start
    path = [current]
    expanded = 0
    while current != goal:
        neighbors = list(graph[current].keys())
        if not neighbors:
            break
        next_node = min(neighbors, key=lambda n: heuristic[n])
        expanded += 1
        if heuristic[next_node] >= heuristic[current]:
            break
        path.append(next_node)
        current = next_node
    exec_time = time.time() - start_time
    return path, len(path)-1, expanded, exec_time
```

Minimax with Alpha-Beta

```
import math

def minimax(node, maximizingPlayer):
    if node == 'CriticalAsset':
        return 0
    if maximizingPlayer:
        best = -math.inf
        for child, cost in graph[node].items():
            val = minimax(child, False) + cost
            best = max(best, val)
        return best
    else:
        best = math.inf
        for child, cost in graph[node].items():
            val = minimax(child, True) + cost
            best = min(best, val)
        return best

def alpha_beta(node, alpha, beta, maximizingPlayer):
    if node == 'CriticalAsset':
        return 0
    if maximizingPlayer:
        best = -math.inf
        for child, cost in graph[node].items():
            val = alpha_beta(child, alpha, beta, False) + cost
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = math.inf
        for child, cost in graph[node].items():
            val = alpha_beta(child, alpha, beta, True) + cost
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best
```


Comparative Results Table

```
import pandas as pd

results = []

def run_all():
    path, cost, expanded, exec_time = bfs(graph, start_node, goal_node)
    results.append({"Algorithm":"BFS","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)"

    path, cost, expanded, exec_time = dfs(graph, start_node, goal_node)
    results.append({"Algorithm":"DFS","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)"

    path, cost, expanded, exec_time = ucs(graph, start_node, goal_node)
    results.append({"Algorithm":"UCS","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)"

    path, cost, expanded, exec_time = a_star(graph, start_node, goal_node)
    results.append({"Algorithm":"A*","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)":

    path, cost, expanded, exec_time = hill_climbing(graph, start_node, goal_node)
    results.append({"Algorithm":"Hill Climbing","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution

    mm_value = minimax(start_node, True)
    ab_value = alpha_beta(start_node, -math.inf, math.inf, True)
    results.append({"Algorithm":"Minimax","Path":"N/A","Total Cost":mm_value,"Nodes Expanded":"-","Execution Time (ms)":"-"})
    results.append({"Algorithm":"Alpha-Beta","Path":"N/A","Total Cost":ab_value,"Nodes Expanded":"-","Execution Time (ms)":"-"}

    df = pd.DataFrame(results)
    print(df.to_string(index=False))

run_all()
```

```
    Algorithm                                        Path  Total Cost Nodes Expanded Execution Time (ms)
          BFS Workstation → Firewall → Server → CriticalAsset           3              7              0.0196
```

```
            DFS Workstation → Router → Database → CriticalAsset          3             4             0.0048
            UCS Workstation → Firewall → Server → CriticalAsset          8             6             0.0212
             A* Workstation → Firewall → Server → CriticalAsset          8             5             0.0136
Hill Climbing Workstation → Router → Database → CriticalAsset          3             3             0.0165
      Minimax                                             N/A         10             -                  -
   Alpha-Beta                                             N/A         10             -                  -
```

Show command palette (Ctrl+Shift+P)

ux design

```
import ipywidgets as widgets
from IPython.display import display
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

results = []

def run_all_algorithms():
    results.clear()
    path, cost, expanded, exec_time = bfs(graph, start_node, goal_node)
    results.append({"Algorithm":"BFS","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)":r

    path, cost, expanded, exec_time = dfs(graph, start_node, goal_node)
    results.append({"Algorithm":"DFS","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)":r

    path, cost, expanded, exec_time = ucs(graph, start_node, goal_node)
    results.append({"Algorithm":"UCS","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)":r

    path, cost, expanded, exec_time = a_star(graph, start_node, goal_node)
    results.append({"Algorithm":"A*","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution Time (ms)":re

    path, cost, expanded, exec_time = hill_climbing(graph, start_node, goal_node)
    results.append({"Algorithm":"Hill Climbing","Path":" → ".join(path),"Total Cost":cost,"Nodes Expanded":expanded,"Execution T

    return pd.DataFrame(results)

def show_graph():
    G = nx.DiGraph()
    for node, neighbors in graph.items():
        for neighbor, weight in neighbors.items():
            G.add_edge(node, neighbor, weight=weight)
    pos = nx.spring_layout(G, seed=42)
    plt.figure(figsize=(8,6))
    nx.draw(G, pos, with_labels=True, node_color="skyblue", node_size=2000, font_size=10, font_weight="bold", arrows=True)
    labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
    plt.title("Network Graph Visualization")
    plt.show()


# --- Dropdown for user choice ---
algo_dropdown = widgets.Dropdown(
    options=["BFS","DFS","UCS","A*","Hill Climbing","Comparison Table","Graph Visualization"],
    value="BFS",
    description="Choose:"
)

output = widgets.Output()

def on_button_click(b):
    with output:
        output.clear_output()
        choice = algo_dropdown.value
        if choice == "Comparison Table":
            df = run_all_algorithms()
            display(df)
        elif choice == "Graph Visualization":
            show_graph()
        else:
            if choice == "BFS":
                path, cost, expanded, exec_time = bfs(graph, start_node, goal_node)
            elif choice == "DFS":
                path, cost, expanded, exec_time = dfs(graph, start_node, goal_node)
            elif choice == "UCS":
                path, cost, expanded, exec_time = ucs(graph, start_node, goal_node)
            elif choice == "A*":
                path, cost, expanded, exec_time = a_star(graph, start_node, goal_node)
```

```
            elif choice == "Hill Climbing":
                path, cost, expanded, exec_time = hill_climbing(graph, start_node, goal_node)

            print(f"Algorithm: {choice}")
            print(f"Path: {' → '.join(path)}")
            print(f"Total Cost: {cost}")
            print(f"Nodes Expanded: {expanded}")
            print(f"Execution Time: {exec_time:.4f}s")

button = widgets.Button(description="Run")
button.on_click(on_button_click)

display(algo_dropdown, button, output)
```


Choose: Comparison Table

Run

	Algorithm	Path	Total Cost	Nodes Expanded	Execution Time (ms)
0	BFS	Workstation → Firewall → Server → CriticalAsset	3	7	0.0153
1	DFS	Workstation → Router → Database → CriticalAsset	3	4	0.0260
2	UCS	Workstation → Firewall → Server → CriticalAsset	8	6	0.0150
3	A*	Workstation → Firewall → Server → CriticalAsset	8	5	0.0107
4	Hill Climbing	Workstation → Router → Database → CriticalAsset	3	3	0.0153